

Manual Técnico

El Zapatito

Sistema de Gestión y Tienda de Calzado en Línea

1. Descripción General del Sistema

"El Zapatito" es una aplicación web de comercio electrónico desarrollada para la gestión y venta de calzado en línea. El sistema cuenta con dos flujos principales:

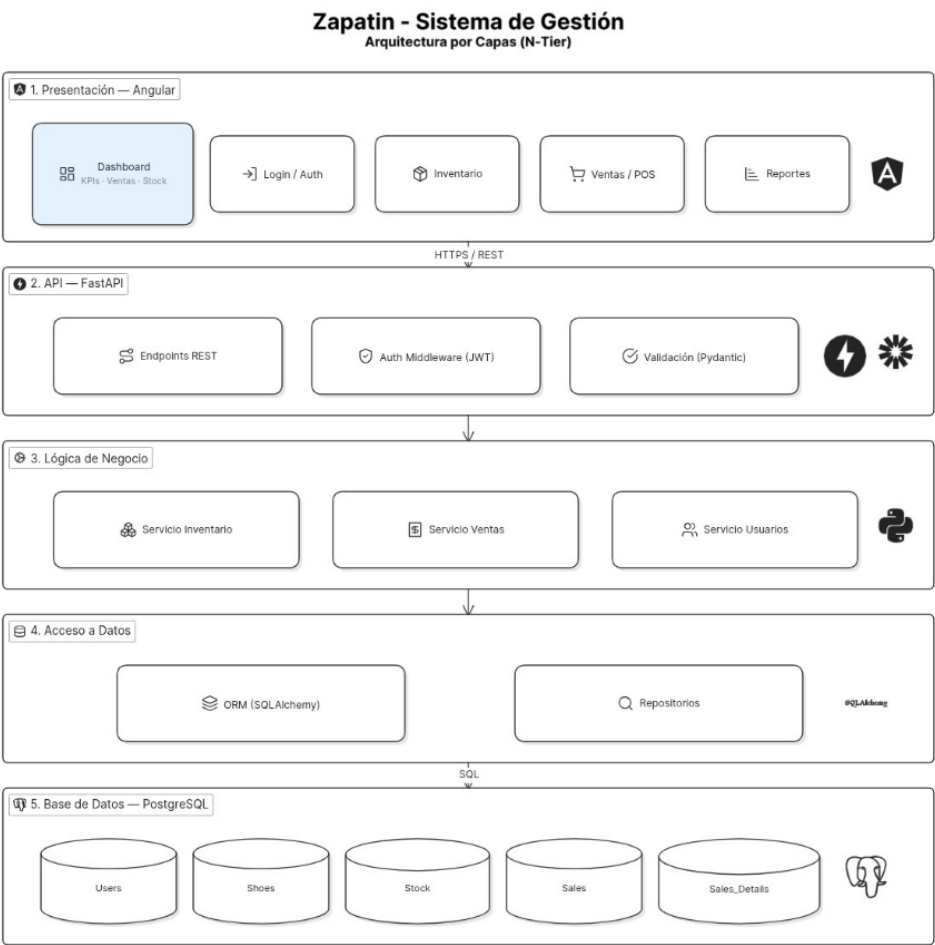
- Flujo del Cliente: Catálogo de productos con filtros, carrito de compras, checkout y pago seguro mediante PayPal.
- Flujo del Administrador: Panel de control con métricas de ventas, gestión completa de productos, inventario por talla, marcas, categorías y cupones de descuento.

Tecnologías principales

- Frontend: Angular 21 (TypeScript) — desplegado en Vercel
- Backend: FastAPI (Python 3.13) — desplegado en Render
- Base de Datos: PostgreSQL en Supabase

2. Arquitectura del Sistema

El sistema implementa una Arquitectura por Capas (N-Tier / Layered Architecture) con 5 niveles bien diferenciados:



Capa 1: Presentación — Angular (Frontend)

Interfaz de usuario que gestiona las vistas del cliente (catálogo, carrito, pago) y del administrador (dashboard, inventario, cupones). Se comunica con la Capa 2 exclusivamente mediante peticiones HTTPS/REST.

Capa 2: API — FastAPI

Puerta de entrada del servidor. Expone los endpoints REST, verifica los tokens JWT en cada petición protegida y valida los datos de entrada usando esquemas Pydantic antes de pasar a la lógica de negocio.

Capa 3: Lógica de Negocio — Servicios Python

Contiene las reglas del negocio: validación de stock, aplicación de cupones, coordinación con PayPal y gestión de sesiones de usuarios. Es la capa que "decide" qué hacer con los datos.

Capa 4: Acceso a Datos — SQLAlchemy (ORM)

Abstrae el acceso a la base de datos usando modelos Python. SQLAlchemy traduce las operaciones de objetos a consultas SQL y gestiona el pool de conexiones.

Capa 5: Base de Datos — PostgreSQL (Supabase)

Capa de persistencia. Almacena todos los datos del sistema: usuarios, productos, inventario, órdenes y cupones de forma relacional y permanente.

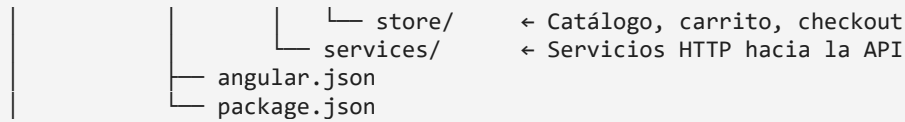
Flujo de una petición completa

```
[Usuario] → Angular (HTTPS) → FastAPI (JWT + Pydantic) → Servicios Python → SQLAlchemy (SQL) → PostgreSQL
```

3. Estructura del Proyecto

El repositorio web_integral es un Monorepo que contiene tanto el frontend como el backend en un mismo repositorio de GitHub.





4. Requisitos Previos e Instalación

Requisitos del Sistema

Herramienta	Versión mínima requerida	Verificar con
Node.js	20.x LTS o superior	node --version
npm	10.x o superior	npm --version
Python	3.11 o superior	python --version
Angular CLI	21.x	ng version
Git	Cualquier versión reciente	git --version

4.1 Clonar el Repositorio

```
git clone https://github.com/PablitinGlez/web_integral.git
cd web_integral
```

4.2 Instalación del Backend

```
# 1. Navegar a la carpeta del backend
cd cliente/codigo/backend

# 2. Crear el entorno virtual de Python
python -m venv venv

# 3. Activar el entorno virtual
.\venv\Scripts\activate      # Windows (PowerShell)
source venv/bin/activate     # Linux / macOS

# 4. Instalar todas las dependencias
pip install -r requirements.txt

# 5. Crear el archivo de variables de entorno
# (copiar el archivo de ejemplo y llenar los valores)
copy .env.example .env      # Windows
cp .env.example .env        # Linux / macOS
```

Nota:

Ver la sección 6 de este manual para el detalle de cada variable de entorno que debe configurarse en el archivo .env.

4.3 Instalación del Frontend

```
# 1. Navegar a la carpeta del frontend
cd cliente/codigo/el-zapatito

# 2. Instalar las dependencias de Node.js
```

```
npm install
```

4.4 Poblar la Base de Datos con Datos de Prueba (Opcional)

Una vez configurado el .env con las credenciales de Supabase, se puede ejecutar el script de semillas para cargar 50 productos, 19 marcas y 10 categorías automáticamente:

```
# Desde la carpeta backend (con el venv activado)
python seed.py
```

Salida esperada:

```
Iniciando semillero de datos...
Tablas limpiadas exitosamente.
Creadas 10 categorías.
Creadas 19 marcas.
Creando 50 productos de prueba...
Base de datos sembrada con éxito!
```

5. Configuración del Servidor

5.1 Servidor de Desarrollo Local

Backend (FastAPI):

```
# Desde cliente/codigo/backend (con venv activado)
uvicorn main:app --reload
```

El servidor estará disponible en <http://127.0.0.1:8000> — la documentación Swagger interactiva estará en <http://127.0.0.1:8000/docs>

Frontend (Angular):

```
# Desde cliente/codigo/el-zapatito
ng serve
```

La aplicación estará disponible en <http://localhost:4200>

5.2 Configuración del Servidor de Producción

Backend en Render

Parámetro	Valor
Plataforma	Render.com
Runtime	Python 3
Root Directory	cliente/codigo/backend
Build Command	pip install -r requirements.txt
Start Command	uvicorn main:app --host 0.0.0.0
URL de producción	https://web-integral.onrender.com

Importante:	En Render, las Variables de Entorno se configuran directamente en el panel de Settings del servicio. No se sube el archivo .env al servidor.
--------------------	--

Importante:	En Render, las Variables de Entorno se configuran directamente en el panel de Settings del servicio. No se sube el archivo .env al servidor.
--------------------	--

Frontend en Vercel

Parámetro	Valor
Plataforma	Vercel.com
Framework	Angular (detectado automáticamente)
Root Directory	cliente/codigo/el-zapato
Build Command	ng build
Output Directory	dist/el-zapato/browser
URL de producción	https://web-integral-wheat.vercel.app

Importante:	El Output Directory debe configurarse manualmente en Vercel como dist/el-zapatico/browser porque Angular 17+ coloca los archivos compilados en esa subcarpeta.
--------------------	--

Importante:	El Output Directory debe configurarse manualmente en Vercel como dist/el-zapatico/browser porque Angular 17+ coloca los archivos compilados en esa subcarpeta.
--------------------	--

5.3 Configuración de CORS

Para que el frontend en Vercel pueda comunicarse con el backend en Render, se configuraron los orígenes permitidos en main.py:

```
app.add_middleware(
    CORSMiddleware,
    allow_origins=[
        "http://localhost:4200",
        "https://web-integral.vercel.app",
        "https://web-integral-wheat.vercel.app"
    ],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

6. Variables de Entorno

El backend utiliza un archivo `.env` que no debe subirse a GitHub (está en el `.gitignore`). Debe crearse manualmente en `cliente/codigo/backend/.env`.

Contenido del archivo .env

```
# — Supabase —————
SUPABASE_URL=https://tu-proyecto.supabase.co
SUPABASE_ANON_KEY=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXbz...
SUPABASE_SERVICE_ROLE_KEY=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXbz...
SUPABASE_JWT_SECRET=tu-jwt-secret

# — Base de Datos (PostgreSQL) —————
# Usar el Pooler IPv4 de Supabase para evitar problemas de red
```

```
DATABASE_URL=postgresql://postgres.xxx:password@aws-1-us-east-1.pooler.supabase.com:5432/postgres
```

```
# — Cloudinary (Imágenes) —————  
CLOUDINARY_CLOUD_NAME=tu-cloud-name  
CLOUDINARY_API_KEY=tu-api-key  
CLOUDINARY_API_SECRET=tu-api-secret
```

```
# — Autenticación JWT —————  
SECRET_KEY=tu-clave-secreta-larga-y-segura  
ALGORITHM=HS256  
ACCESS_TOKEN_EXPIRE_MINUTES=60
```

```
# — PayPal Sandbox —————  
PAYPAL_CLIENT_ID=tu-paypal-client-id-sandbox  
PAYPAL_CLIENT_SECRET=tu-paypal-client-secret-sandbox
```

Descripción de Variables

Variable	Dónde obtenerla
SUPABASE_URL	Panel de Supabase → Settings → API → Project URL
SUPABASE_ANON_KEY	Panel de Supabase → Settings → API → anon public
SUPABASE_SERVICE_ROLE_KEY	Panel de Supabase → Settings → API → service_role
SUPABASE_JWT_SECRET	Panel de Supabase → Settings → API → JWT Settings
DATABASE_URL	Panel de Supabase → Settings → Database → Connection Pooling (Mode: Transaction, IPv4)
CLOUDINARY_CLOUD_NAME	Panel de Cloudinary → Dashboard → Cloud name
CLOUDINARY_API_KEY	Panel de Cloudinary → Dashboard → API Key
CLOUDINARY_API_SECRET	Panel de Cloudinary → Dashboard → API Secret
PAYPAL_CLIENT_ID	developer.paypal.com → My Apps → Sandbox → Client ID
PAYPAL_CLIENT_SECRET	developer.paypal.com → My Apps → Sandbox → Secret

7. Base de Datos

El sistema utiliza PostgreSQL alojado en Supabase. Las tablas se crean automáticamente al iniciar el backend gracias a SQLAlchemy (`Base.metadata.create_all(bind=engine)` en `main.py`).

categories			
UUID	id		PK
VARCHAR	name		
TEXT	description		
TIMESTAMP	created_at		

users			
UUID	id		PK
VARCHAR	email		
TEXT	hashed_password		
VARCHAR	full_name		
VARCHAR	role		
BOOLEAN	is_active		
TIMESTAMP	created_at		

products			
UUID	id		PK
VARCHAR	name		
VARCHAR	brand		
TEXT	description		
DECIMAL	price		
DECIMAL	base_price		
UUID	category_id		FK
TEXT	image_url		
BOOLEAN	is_active		
TIMESTAMP	created_at		
TIMESTAMP	updated_at		

discounts			
UUID	id		PK
VARCHAR	name		
VARCHAR	discount_type		
DECIMAL	value		
TIMESTAMP	start_date		
TIMESTAMP	end_date		
BOOLEAN	is_active		
TIMESTAMP	created_at		

inventory			
UUID	id		PK
UUID	product_id		FK
DECIMAL	size		
INTEGER	stock_quantity		

product_discounts			
UUID	product_id		PK,FK
UUID	discount_id		PK,FK

orders			
UUID	id		PK
DECIMAL	total_amount		
VARCHAR	status		
VARCHAR	customer_email		
TIMESTAMP	created_at		

order_items			
UUID	id		PK
UUID	order_id		FK
UUID	inventory_id		FK
INTEGER	quantity		
DECIMAL	price_at_time		

tiene

tiene variantes

aplica

aplica a

referenciado en

contiene

}

Tablas del Sistema

Tabla	Descripción
users	Usuarios registrados (gestionada por Supabase Auth)
categories	Categorías de clasificación del calzado
brands	Marcas disponibles en la tienda
products	Catálogo completo de calzado
product_images	Galería de imágenes adicionales por producto
inventory	Stock físico desglosado por talla para cada producto
addresses	Direcciones de envío por usuario
orders	Historial de órdenes con folio de PayPal y estatus
order_items	Detalle línea por línea de cada orden
coupons	Códigos de descuento con reglas de vigencia

Conexión a la Base de Datos

La conexión se establece en `app/database.py` usando SQLAlchemy:

```
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker
from app.core.config import settings

engine = create_engine(settings.DATABASE_URL)
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
```

Nota técnica:

Se recomienda usar el Connection Pooler en modo Transaction (IPv4) de Supabase como DATABASE_URL. Esto garantiza compatibilidad con redes residenciales en México (que no soportan IPv6) y mejora el rendimiento bajo concurrencia.

8. Dependencias

Backend (Python) — requirements.txt

Paquete	Función
fastapi	Framework principal del servidor API REST
uvicorn	Servidor ASGI para ejecutar FastAPI
sqlalchemy	ORM para gestionar la base de datos relacional
psycopg2-binary	Driver de conexión entre SQLAlchemy y PostgreSQL
python-dotenv	Carga variables de entorno desde el archivo .env
pydantic[email]	Validación de datos y correos electrónicos
pydantic-settings	Gestión tipada de configuración desde variables de entorno
python-jose[cryptography]	Generación y verificación de tokens JWT

Paquete	Función
passlib[bcrypt]	Encriptación segura de contraseñas con bcrypt
python-multipart	Soporte para subida de archivos e imágenes
cloudinary	SDK para gestión de imágenes en Cloudinary

Frontend (Node.js) — package.json

Paquete	Función
@angular/core	Núcleo del framework Angular
@angular/router	Sistema de enrutamiento y navegación
@angular/forms	Formularios reactivos (login, checkout, admin)
@angular/common/http	Módulo HTTP para consumir la API REST
@supabase/supabase-js	Cliente de Supabase para autenticación
rxjs	Manejo de operaciones asíncronas y observables
typescript	Lenguaje de programación tipado (dev)
@angular/cli	CLI para compilar y servir la aplicación (dev)